

# TurnKV: Turn-Aware Cache Eviction for Multi-Turn LLM Conversations

Gaurav Shah (gshah2@andrew.cmu.edu) Pradyut Ganesh (pganesh@andrew.cmu.edu) Yagneek Patlola (ypatlola@andrew.cmu.edu)  
Carnegie Mellon University

## 1. Introduction

Modern LLM deployments are predominantly conversational, with dialogue histories that grow turn by turn and place sustained pressure on GPU memory. The KV cache (storing key and value tensors for every prior token) is the dominant memory bottleneck, growing linearly with context length. This has motivated work on KV cache eviction, where tokens deemed unimportant are permanently discarded. Existing methods perform well on single-turn benchmarks (one document, one question), but multi-turn conversations are structurally different: they require policies that are explicitly aware of conversation structure, not just the current query.

## 2. Problem

All major eviction methods, including H2O [1], SnapKV [2], PyramidKV [3], and StreamingLLM [4], score tokens using attention weights from the *current* query window, then permanently remove the lowest-scoring ones. In single-turn settings this is reasonable, but in multi-turn conversations tokens from an early turn that are irrelevant now may be critical two turns later; once evicted, they cannot be recovered. Formally, standard methods compute  $s_i \propto A(Q_{\text{window}}, K_i)$ , which has no dependence on which turn produced token  $i$  or what future queries may require. Our research question: **can a scoring function that encodes conversation structure improve eviction quality in multi-turn settings without training or external models?**

## 3. Related Work

**Standard eviction methods.** SnapKV clusters attention weights over a trailing window; PyramidKV allocates smaller budgets to deeper layers; H2O retains tokens by accumulated attention; StreamingLLM keeps only sink and recent tokens. None encode turn boundaries. NVIDIA’s **kvpress** [5] consolidates over twenty such methods under a unified **score()** interface and serves as our development and evaluation platform.

**Evidence of the failure.** SCBench [6] (ICLR 2025) evaluates KV cache methods under multi-turn conditions across 931 sessions and twelve task types, finding that eviction-based methods degrade consistently as turn count increases while full-cache methods do not. LongMemEval [7] (ICLR 2025) evaluates long-term chat memory over  $\sim 115\text{k}$ -token histories, where state-of-the-art commercial systems achieve only 30–70% accuracy. No existing press in **kvpress** targets turn-structured input, and neither benchmark appears in its evaluation suite.

**The only multi-turn-aware method.** EpiCache [8] (Apple, 2025) clusters conversation history into topical episodes using a sentence encoder and K-means, achieving up to 40% accuracy improvement over eviction baselines. However, it requires an offline clustering phase with an external embedding model before any query arrives, precluding online use, and it is not integrated with any standard inference framework. Our methods target this gap: online, training-free, and natively compatible with **kvpress**.

## 4. Implementation Plan

**Proposed policies.** We implement three new **Press** classes in **kvpress** by inheriting from **ScorerPress** or **BasePress**, requiring no training and no external models.

(1) *Cross-Turn Accumulation Press.* We maintain a running score tensor updated at each turn boundary by accumulating attention weights from all past turns’ queries. Rather than raw summation (which makes old tokens permanently unevictable), we apply exponential decay:  $s_i \leftarrow \gamma \cdot s_i + a_i^{\text{new}}$ ,  $\gamma \approx 0.9$ , so tokens that stop being attended to fade naturally. To handle topic shifts, we compute cosine similarity between mean key vectors of consecutive turns; a drop below a threshold triggers a decay pulse on accumulated scores, requiring only one dot product per turn with no external model.

(2) *Turn-Floor Press.* We enforce a minimum token budget per turn so that no early turn is fully evicted. A uniform floor collapses as turn count grows and wastes budget on filler turns, so we use a length-proportional floor:  $B_k = \max(\alpha \cdot B_{\text{total}} \cdot |T_k| / \sum_j |T_j|, c)$ , where  $c$  is a hard per-turn minimum (e.g., 5 tokens). Turns shorter than a threshold  $m$  (e.g., single-sentence acknowledgments) are exempt from floor protection. Turn boundaries are identified from role-marker tokens detected via the tokenizer’s **apply\_chat\_template()** at initialization, avoiding hard-coded template markers.

(3) *Role-Boundary Anchor Press.* The tokens near each role boundary encode the core intent of a turn yet are systematically undervalued by trailing-window attention scoring. We add a fixed retention bonus to a window of  $w_k = \max(3, \lfloor \beta |T_k| \rfloor)$  tokens (with  $\beta \approx 0.15$ ) around each boundary. Since intent often appears at the end of user messages while assistant openings are frequently boilerplate, we apply a dual anchor (first and last  $w_k$  tokens) to user turns and a tail-only anchor to assistant turns.

**Composition.** The policies compose naturally: the floor guarantees a minimum per-turn budget; the anchor bonus is added to boundary token scores before ranking; and the decayed cross-turn score drives remaining eviction. We report all seven ablation combinations on SCBench.

**Datasets, baselines, and evaluation.** Primary: **SCBench** (multi-turn retrieval, QA, multi-hop); stress test: **Long-MemEval-S** ( $\sim 115\text{k}$ -token episodic histories); regression guard: **LongBench** (single-turn). Model: Llama-3.1-8B-Instruct. Baselines from **kvpress**: SnapKVPress, ObservedAttentionPress (H2O), StreamingLLMPress, KnormPress, ExpectedAttentionPress. We extend **kvpress**’s CLI with a multi-turn harness that processes sessions turn-by-turn and re-applies the press at each boundary while preserving the prior KV cache. Primary metric: *accuracy-per-turn-number*, alongside aggregate accuracy, peak memory, and decoding latency.

**References** [1] Z. Zhang et al., “H<sub>2</sub>O,” NeurIPS 2023. [2] Y. Li et al., “SnapKV,” NeurIPS 2024. [3] C. Cai et al., “PyramidKV,” arXiv:2406.02069, 2024. [4] G. Xiao et al., “StreamingLLM,” ICLR 2024. [5] A. Devoto et al., “kvpress,” arXiv:2510.00636, 2025. [6] Y. Li et al., “SCBench,” ICLR 2025. [7] D. Wu et al., “LongMemEval,” ICLR 2025. [8] M. Kim et al., “EpiCache,” arXiv:2509.17396, 2025.